Gemelo digital del brazo XK-JS1-1

Arquitectura, desarrollo y funcionamiento

ETN-902 / ETN-1034 / ETN-1000 · UMSA · II-2026

28 de octubre de 2026

Resumen

Este documento explica *cómo está construido* el gemelo digital del brazo XK-JS1-1 y *por qué* se hizo así. Complementa al *Manual de conexión y uso*, que explica cómo usarlo. Se describen las etapas del desarrollo, la arquitectura de software, las páginas web y sus puertas de acceso, el protocolo PPI con el que la PC habla con el PLC S7-200, el programa nuevo del PLC y el funcionamiento detallado de los tres modos de mando: articular, cartesiano y secuencias.

Índice

1. De dónde partimos **3**

2. Cómo se desarrolló **3**

3. Arquitectura general **4**
3.1. Un ciclo de lectura . . . 5
3.2. Una orden . . . 5

4. Las páginas web **6**
4.1. Cuáles son . . . 6
4.2. ¿Por qué una puerta local y otra de red si todo está en la misma red? . . . 6
4.3. Cómo funciona el ingreso . . . 7

5. El protocolo PPI **8**
5.1. Qué es . . . 8
5.2. Capa física: RS-485 y el cable USB-PPI . . . 8
5.3. Cada byte: 11 bits . . . 8
5.4. Las tramas . . . 9
5.5. Un intercambio completo . . . 9
5.6. El mensaje S7 por dentro: un ejemplo real . . . 10
5.7. Lo que se aprendió en la planta . . . 10

6. El programa nuevo del PLC **11**
6.1. Un solo tren de pulsos para X e Y . . . 12

7. Los modos de mando **12**
7.1. Modo articular . . . 12
7.2. Modo cartesiano . . . 13
7.3. Modo secuencias . . . 15

8. Seguridad por capas **16**

9. Limitaciones generales y siguientes pasos **16**

1. De dónde partimos

La planta original es un brazo de 4 grados de libertad (base giratoria, carro X, carro Y y garra) con tres motores paso a paso, drivers JMC 2M542-N y un PLC S7-200 (CPU 224 CN). El PLC tenía un programa de **secuencias fijas**: un homing y dos rutinas grabadas.

Planta original	Qué le faltaba
El PLC manda pulsos, pero no guarda la posición	Nadie sabía dónde estaba el brazo: no había número de posición, solo finales de carrera
Rutinas fijas programadas en Micro/WIN	Para llevar la garra a otro punto había que reprogramar el PLC y calcular los pulsos a mano
Panel con pulsadores y selector	No había forma de ver ni de mandar el brazo desde una PC, y menos desde otro lugar
Sin modelo matemático	No se conocía qué puntos alcanza la garra ni cómo se relacionan los carros con la posición de la garra

El objetivo fue construir un **gemelo digital**: un modelo del brazo en la computadora que se mueve igual que el brazo real, lo muestra en vivo y permite mandarlo a cualquier punto, desde la PC del laboratorio o desde otros equipos de la red.

2. Cómo se desarrolló

El trabajo avanzó por capas: cada etapa se apoya en la anterior.

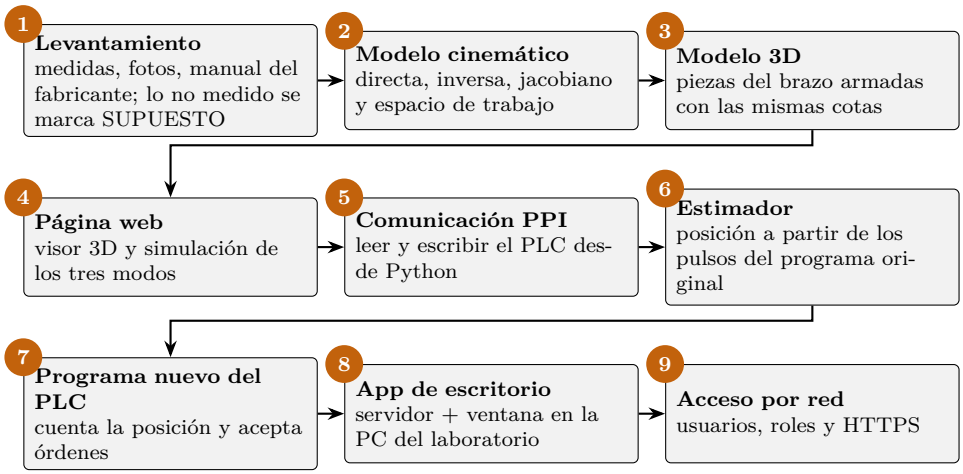


Figura 1: Etapas del desarrollo.

1. **Levantamiento.** Se midieron las piezas (bocetos, fotos del brazo y del tablero) y se tomaron datos del manual del fabricante. Cada cota quedó marcada como *leída*, *estimada* o *SUPUESTO*, para saber qué hay que verificar en la planta. Del tablero se leyó la configuración de los drivers: 1000 pulsos por vuelta, que con un husillo de 5 mm dan **0,005 mm por pulso**.
2. **Modelo cinemático** (`web/js/cinematica.js`). El brazo es un mecanismo de bielas: el carro X desplaza todo el bastidor y el carro Y mueve un bloque que, a través de una biela, hace girar el brazo. Con geometría plana se escribieron la *cinemática directa* (de carros a garra), la *inversa* (de garra a carros), el jacobiano y el espacio de trabajo. Es la única fuente de parámetros: la usan la página, el modelo 3D y las gráficas.

3. **Modelo 3D.** Las piezas se modelaron y se armaron con las mismas cotas del modelo cinemático, de modo que el dibujo y el cálculo coinciden.
4. **Página web.** Un visor 3D (biblioteca three.js) dibuja el brazo con la pose que calcula la cinemática. Permite simular los tres modos de mando sin hardware.
5. **Comunicación PPI** (`app/ppi.py`). Se programó desde cero, en Python y sin bibliotecas externas, un cliente del protocolo PPI para leer y escribir la memoria del PLC por el cable USB-PPI (sección 5).
6. **Estimador.** Como el programa original no guarda la posición, la app la *estima*: lee qué salidas de pulsos están activas y cuánto tiempo, y la corrige cada vez que se pisa un final de carrera.
7. **Programa nuevo del PLC** (`plc/brazo_gemelo.awl`). Cuenta los pulsos de cada eje (posición exacta), recibe órdenes por un buzón en la memoria V, respeta los finales de carrera y se detiene solo si la PC deja de responder (sección 6).
8. **App de escritorio** (`app/gemelo_brazo.py`). Une todo: habla con el PLC, sirve la página y abre una ventana propia.
9. **Acceso por red.** Se agregaron usuarios con contraseña, roles (observador y operador), conexión cifrada y un único mando a la vez.

3. Arquitectura general

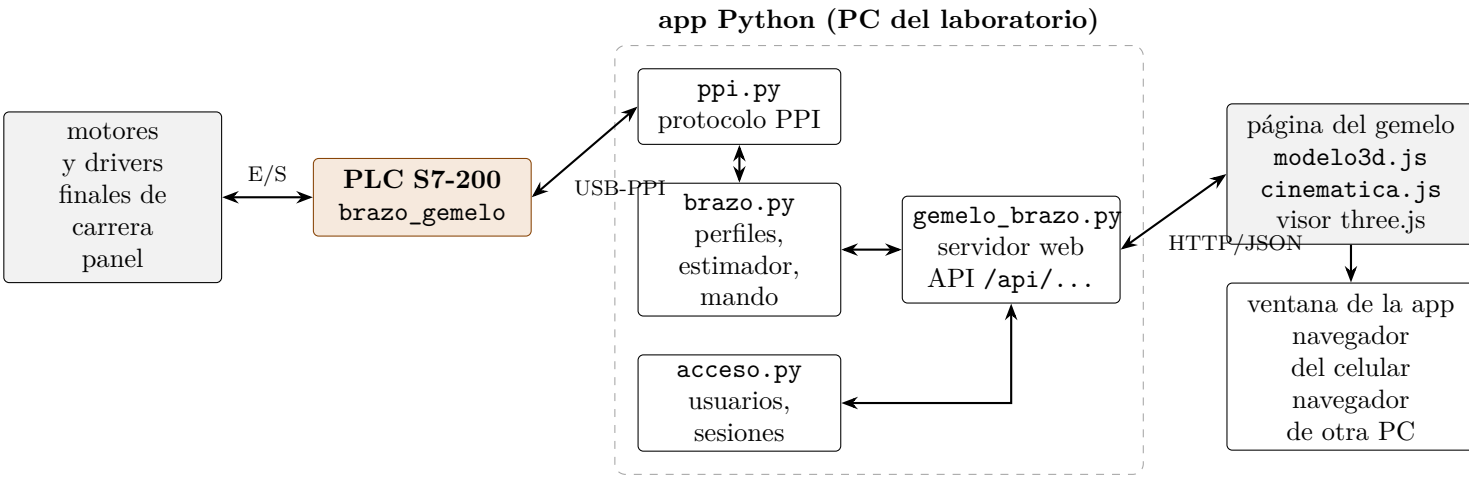


Figura 2: Piezas del sistema y cómo se comunican.

Archivo	Qué hace
`web/js/cinematica.js`	Modelo matemático: parámetros, directa, inversa, jacobiano, conversión a pulsos, trayectorias.
`web/js/modelo3d.js`	Visor 3D, pestañas de mando, simulación y seguimiento del brazo real.
`web/js/conexion-plc.js`	Habla con la app: modo simulación o brazo real, lectura del estado, órdenes.
`app/ppi.py`	Cliente del protocolo PPI: lecturas y escrituras en la memoria del PLC.
`app/brazo.py`	Reconoce el programa del PLC (perfiles), lee el estado en un bucle, traduce órdenes a escrituras, estima la posición y lleva quién tiene el mando.
`app/gemelo_brazo.py`	Servidor web: sirve la página, la API, el ingreso y el acceso por red.
`app/acceso.py`, `usuarios.py`	Contraseñas cifradas, sesiones, bloqueo por intentos; herramienta para crear usuarios.
`app/ventana.py`	Ventana propia (GTK + WebKit) que muestra la página local.
`plc/brazo_gemelo.awl`	Programa nuevo del PLC.

3.1. Un ciclo de lectura

La app tiene un **bucle** que pregunta al PLC por su estado tan rápido como permite el cable (unas 6 veces por segundo a 9,6 kbit/s). El resultado queda en memoria. Cada navegador, por su lado, pide ese estado a la app (`GET /api/estado`) y el visor *interpola* entre dos lecturas, con un pequeño retardo (0,1 a 0,5 s), para que el movimiento se vea suave y no a saltos.

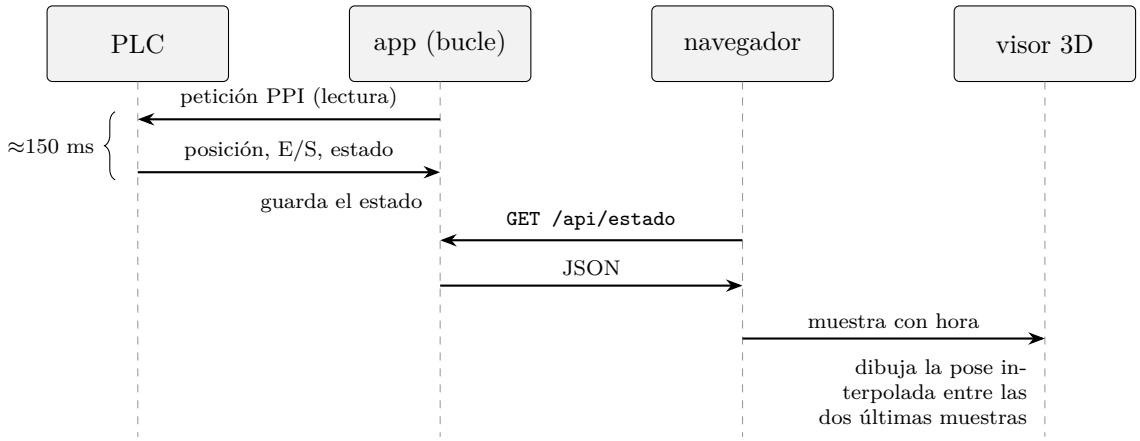


Figura 3: Del PLC al dibujo: el bucle de la app y el sondeo del navegador corren por separado.

3.2. Una orden

Cuando alguien pulsa *Mover*, la orden atraviesa varios filtros antes de llegar al PLC. Si alguno falla, la orden se rechaza, se le explica el motivo al usuario y queda anotada en el registro.

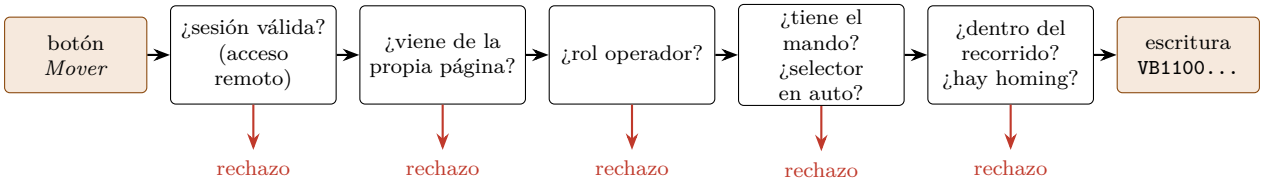


Figura 4: Filtros por los que pasa una orden. El último filtro se repite dentro del PLC, que tiene sus propios límites.

4. Las páginas web

4.1. Cuáles son

En realidad hay **una sola página del gemelo** (los mismos archivos HTML y JavaScript), servida por tres *puertas* distintas, más una página de ingreso.

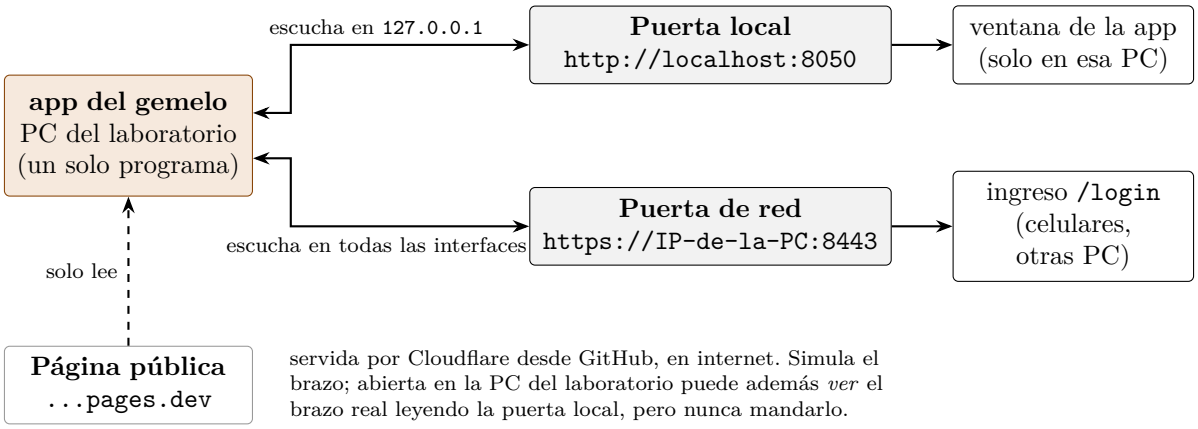


Figura 5: Una misma página, tres puertas.

Puerta	Quién la sirve	Contraseña	Cifrado	Puede mandar el brazo
Local `:8050`	app, PC del laboratorio	no	no hace falta	sí
Red `:8443`	app, PC del laboratorio	sí, con rol	HTTPS	solo operadores con el mando
Pública	Cloudflare (internet)	no	HTTPS	no: simula y, en la PC del laboratorio, puede ver

4.2. ¿Por qué una puerta local y otra de red si todo está en la misma red?

La diferencia no es la página sino **quién puede llegar a ella**. Un servidor elige en qué dirección escucha:

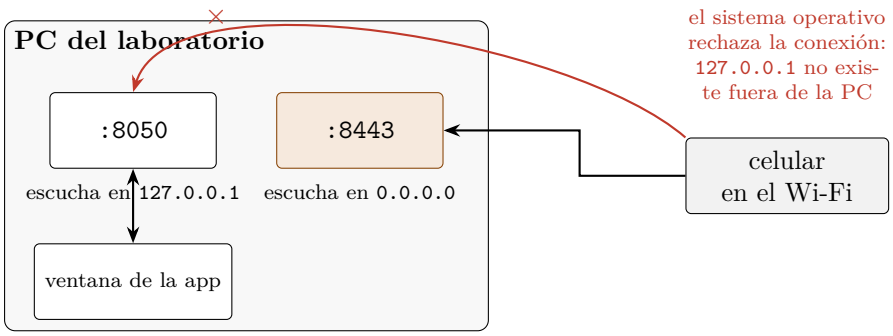


Figura 6: La puerta local es inalcanzable desde la red, aunque los equipos estén en el mismo Wi-Fi.

- **La puerta local solo existe dentro de la PC.** La dirección `127.0.0.1` (*localhost*) no sale a la red: un celular no puede conectarse a ella aunque esté en el mismo Wi-Fi. Por eso no necesita contraseña: quien la usa está sentado frente a la PC del laboratorio, junto al brazo y a la seta de emergencia. Además funciona aunque no haya router ni red.

- **La puerta de red está abierta a todo el Wi-Fi.** Cualquiera conectado a la red puede intentar entrar. Por eso lleva:
  - **HTTPS**: en un Wi-Fi las contraseñas y las órdenes viajan por el aire; sin cifrado, otro equipo de la red podría leerlas o falsificarlas;
  - **usuario, contraseña y rol**, con bloqueo tras 5 intentos fallidos;
  - **un solo mando a la vez**, para que dos operadores no manden el brazo al mismo tiempo;
  - **comprobación de origen**: solo se aceptan órdenes enviadas desde la propia página, no desde otro sitio web abierto en el celular.
- **¿Por qué no usar la puerta de red también en la PC del laboratorio?** Se podría, pero:
  - el certificado es propio del laboratorio y el navegador lo marca como «no seguro»; la ventana de la app tendría que aceptarlo;
  - habría que ingresar con usuario cada vez, incluso estando junto al brazo;
  - si la red falla o cambia la IP, el mando local seguiría dependiendo de ella.

  Separar las puertas mantiene el mando local **simple y siempre disponible**, y concentra todas las protecciones donde hacen falta: en la puerta que da a la red.
- **La página pública** vive en internet (Cloudflare) y no tiene acceso a la red del laboratorio. Los navegadores sí le permiten *leer* `http://localhost` cuando se abre en la propia PC del laboratorio, y la app solo le responde lecturas: por eso puede mostrar el brazo real, pero nunca mandarlo.

4.3. Cómo funciona el ingreso

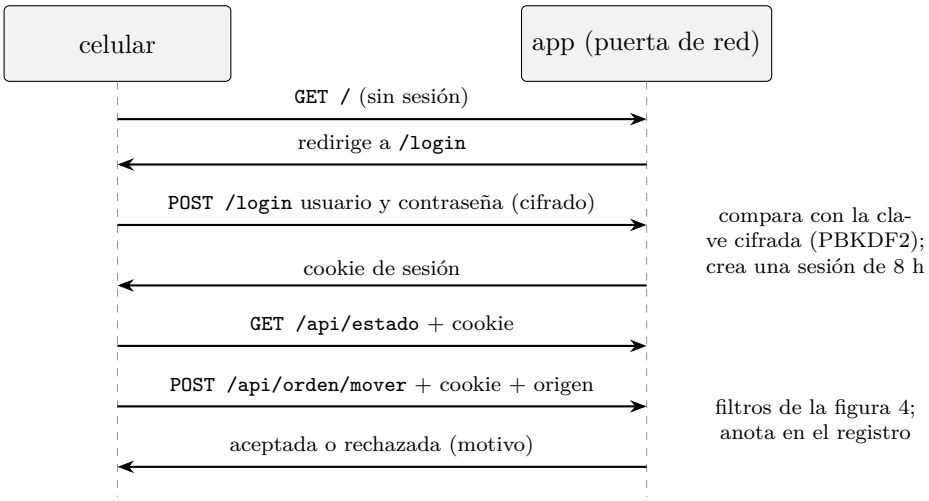


Figura 7: Ingreso y uso de la puerta de red.

Las contraseñas nunca se guardan tal cual: se guarda el resultado de aplicarles 200 000 veces una función de resumen con un valor aleatorio propio de cada usuario (PBKDF2). Así, aunque alguien copie `usuarios.json`, no puede leer las contraseñas.

Limitaciones. La puerta de red llega solo a la red local; para usarla desde internet haría falta una VPN o un túnel. El certificado es autofirmado, por eso el aviso del navegador la primera vez. La PC del laboratorio debe estar encendida y con la app abierta: es la única que habla con el PLC.

5. El protocolo PPI

5.1. Qué es

PPI (*Point-to-Point Interface*) es el protocolo de Siemens con el que el software Micro/WIN programa y monitorea los S7-200. Es un protocolo **maestro–esclavo** sobre RS-485, derivado de PROFIBUS: el maestro (la PC, dirección 0) pregunta y el esclavo (el PLC, hoy dirección 2) responde. Dentro de las tramas PPI viajan los mensajes *S7*, que son los que dicen «lee tal zona de memoria» o «escribe tal valor».

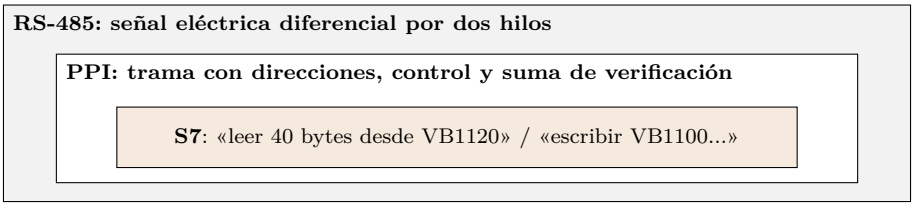


Figura 8: Capas del protocolo: un mensaje S7 dentro de una trama PPI que viaja por RS-485.

5.2. Capa física: RS-485 y el cable USB-PPI

El puerto del S7-200 es un conector DB9 hembra con una línea RS-485. RS-485 transmite cada bit como la *diferencia* de tensión entre dos hilos (A y B): el ruido que afecta a los dos por igual se cancela, lo que la hace robusta en un tablero con motores. Es **semidúplex**: por el mismo par se transmite en un sentido o en el otro, nunca a la vez, así que hablan por turnos.

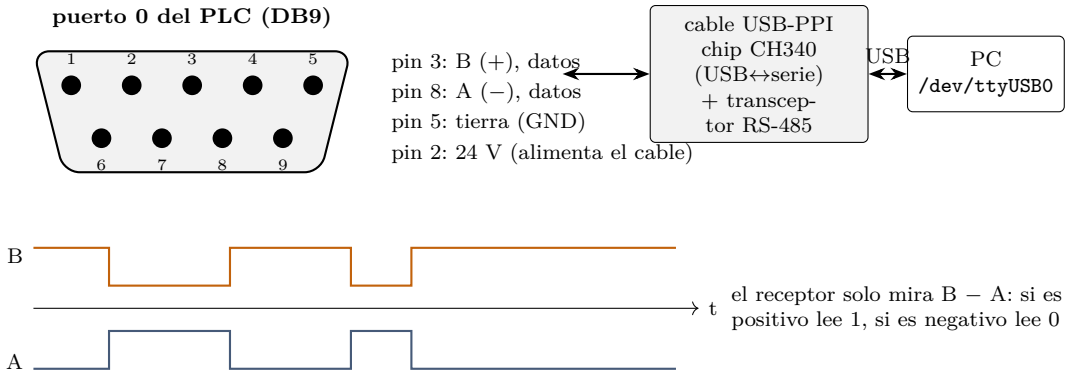


Figura 9: Conector del PLC, cable USB-PPI y señal diferencial RS-485 (esquemática).

Para Linux, el cable es un **puerto serie** más (`/dev/ttyUSB0`): el programa escribe y lee bytes, y el cable se encarga de convertirlos a RS-485 y de cambiar de transmitir a recibir. Por eso un convertidor USB–RS485 genérico, con cambio de dirección automático, debería poder reemplazarlo.

5.3. Cada byte: 11 bits

PPI usa un formato serie de **8 bits de datos, paridad par y 1 bit de parada**: cada byte ocupa 11 bits en la línea.

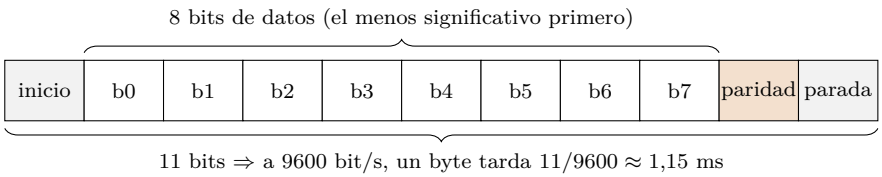


Figura 10: Formato de cada carácter (8E1). La paridad par permite detectar un bit alterado.

El S7-200 admite 9,6, 19,2 y 187,5 kbit/s. El cable USB-PPI del laboratorio trabaja a 9,6 o 19,2. El programa original dejaba el PLC a 19,2 kbit/s (dirección 6); el bloque de sistema del programa nuevo lo dejó a **9,6 kbit/s** (dirección 2). La app prueba las combinaciones habituales y encuentra sola la que corresponde.

5.4. Las tramas

PPI usa tres tipos de trama. Todas las direcciones y la suma de verificación van dentro:

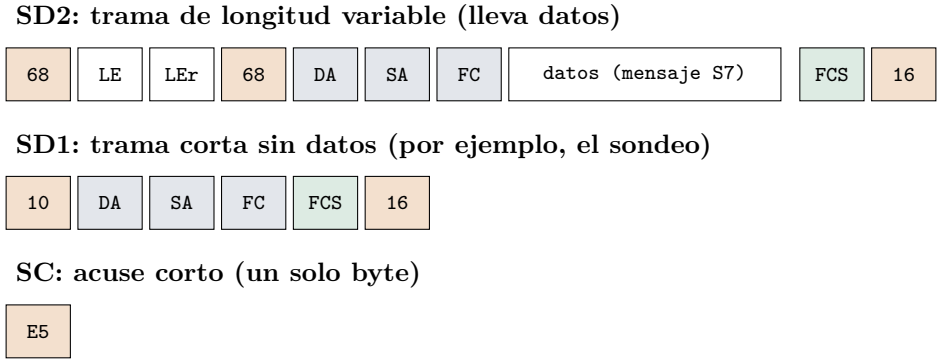


Figura 11: Formatos de trama.

Campo	Significado
`68 / 10`	delimitador de inicio (SD2 o SD1)
`LE, LEr`	longitud de DA + SA + FC + datos, repetida para detectar errores
`DA, SA`	dirección de destino y de origen (PC = 0, PLC = 2)
`FC`	código de función: `6C`/`7C` «petición con respuesta» (el bit FCB alterna para distinguir una trama nueva de una repetida); `5C`/`7C` sondeo «¿tienes mi respuesta?»; `08` respuesta con datos
`FCS`	suma de verificación: suma de DA, SA, FC y los datos, módulo 256
`16`	delimitador de fin
`E5`	acuse corto: «recibí tu trama» (o «todavía no tengo la respuesta»)

5.5. Un intercambio completo

El PLC no contesta con los datos de inmediato: primero acusa recibo y prepara la respuesta mientras sigue con su ciclo. La PC vuelve a preguntar (sondeo) hasta recibirla.

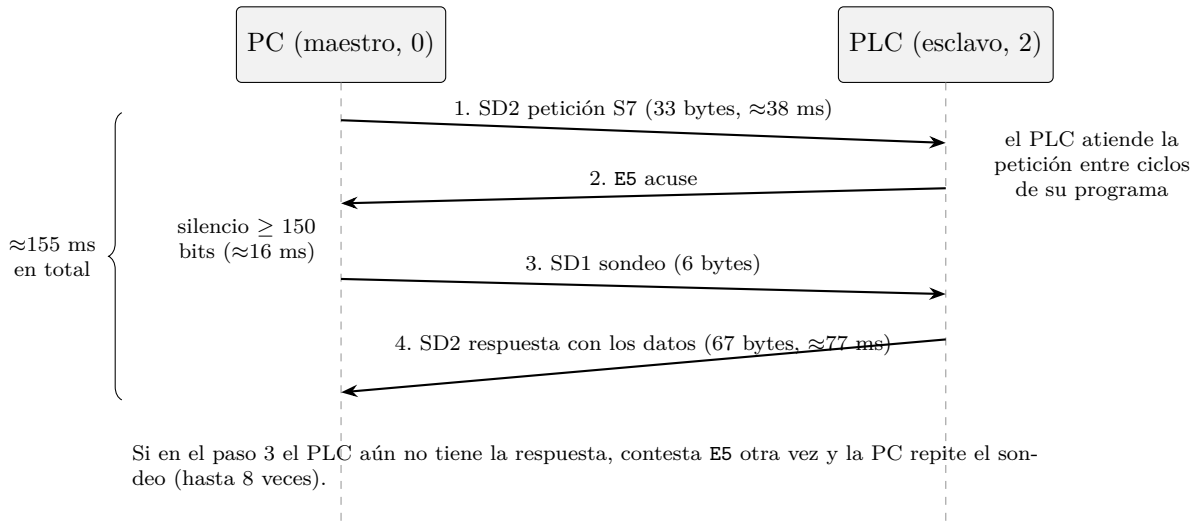


Figura 12: Lectura de 40 bytes a 9,6 kbit/s, con los tiempos medidos en la planta.

5.6. El mensaje S7 por dentro: un ejemplo real

Esta es, byte a byte, la petición que la app envía unas seis veces por segundo para leer el estado del programa nuevo (40 bytes desde `VB1120`):

`68` `1b` `1b` `68` `02` `00` `6c`	**cabecera PPI**: inicio, longitud 27 (`1b`), PLC 2, PC 0, petición (`6c`)
`32` `01` `00` `00` `00` `01` `00` `0e` `00` `00`	**cabecera S7**: `32` protocolo S7, `01` petición, referencia 1, 14 bytes de parámetros, 0 de datos
`04` `01`	**función**: `04` leer, `01` una sola zona
`12` `0a` `10` `02` `00` `28` `00` `01` `84` `00` `23` `00`	**zona**: tamaño byte (`02`), 40 bytes (`0028`), área V (`84`), dirección en bits $1120 \times 8 = 8960$ = `002300`
`b3` `16`	**cierre**: suma de verificación y fin

Figura 13: Petición «leer 40 bytes desde VB1120» (33 bytes en total).

La respuesta (67 bytes) tiene la misma estructura: cabecera PPI con las direcciones invertidas, cabecera S7 de tipo `03` (respuesta, con código de error 0), la función `04 01`, el código `FF` («lectura correcta») y los 40 bytes pedidos: estado, banderas, error, posición de cada eje, entradas, salidas y la firma `47` con la que la app reconoce el programa.

Para escribir (por ejemplo una orden en `VB1100`) se usa la función `05` con los datos al final. La app **solo** permite escribir en zonas fijas (`VB1100-VB1119` con el programa nuevo): cualquier otra dirección se rechaza antes de salir por el cable.

5.7. Lo que se aprendió en la planta

- **El PLC necesita un silencio entre tramas.** Al pasar a 9,6 kbit/s, la app se desconectaba una y otra vez: el PLC ignoraba la trama que llegaba pegada a la anterior. El primer sondeo, enviado

justo después del acuse, se perdía y la app esperaba 0,4 s antes de reintentar. Con un silencio de unos 150 bits antes de transmitir, las lecturas pasaron de ≈570 ms a ≈155 ms y dejaron de fallar.

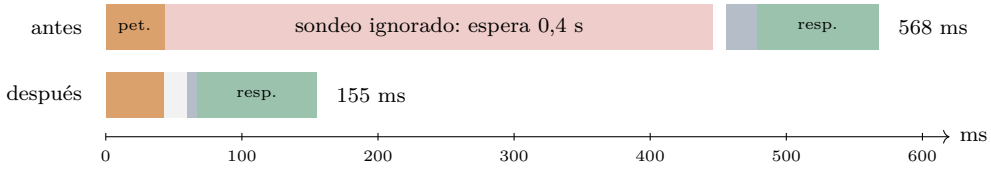


Figura 14: Tiempo de una lectura antes y después de agregar el silencio entre tramas (mediciones reales).

- **Un solo maestro a la vez.** El cable lo usa la app o Micro/WIN, nunca los dos. Por eso, para cargar un programa, hay que cerrar la app y pasar el USB a la máquina virtual.
- **Búsqueda automática.** Si cambia la dirección o la velocidad del PLC (por ejemplo, al cargar un bloque de sistema), la app prueba primero las combinaciones conocidas (2 y 6, a 9,6 y 19,2) y luego las 31 direcciones posibles.

Limitaciones. A 9,6 kbit/s caben unas 6 lecturas por segundo: suficiente para ver el brazo, no para un control rápido en lazo cerrado desde la PC. Por eso los movimientos los ejecuta el PLC, y la PC solo le da el destino y observa. Pasar a 19,2 kbit/s duplicaría la frecuencia de lectura.

6. El programa nuevo del PLC

El programa original no servía para un gemelo: no guardaba la posición y solo sabía hacer sus secuencias. El programa `brazo_gemelo` se escribió con tres ideas:

1. **El PLC cuenta la posición.** Cada tren de pulsos que genera se suma con su signo a un contador por eje. Así la posición es exacta (al pulso) mientras no se pierdan pasos, y la PC solo la lee.
2. **Un buzón de órdenes.** La PC escribe la orden en una zona fija de memoria y cambia un número de secuencia; el PLC la ejecuta una sola vez por cada número nuevo y responde con un eco. Así una orden nunca se ejecuta dos veces por una escritura repetida.
3. **El PLC protege la máquina.** Rechaza destinos fuera del recorrido, corta el movimiento en los finales de carrera y se detiene si la PC deja de dar señales de vida durante 2 s (watchdog).

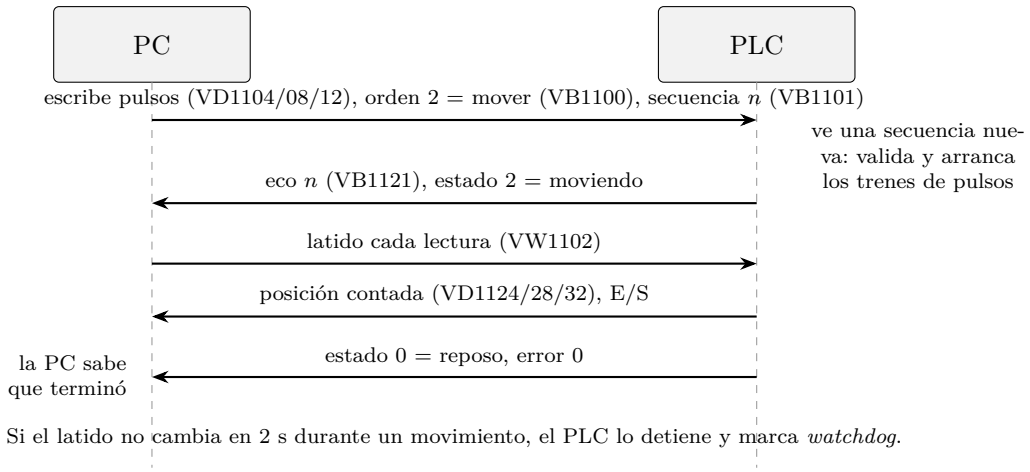


Figura 15: Protocolo del buzón entre la app y el programa `brazo_gemelo`.

6.1. Un solo tren de pulsos para X e Y

La planta tiene una limitación de cableado: el PLC solo tiene dos salidas de pulsos rápidos (Q0.0 y Q0.1). Q0.0 va a la base y **Q0.1 va a los dos carros**, repartida por dos relés: K1 (Q0.5) conecta el carro X y K2 (Q0.6) el carro Y. Cada carro tiene su propia salida de dirección.

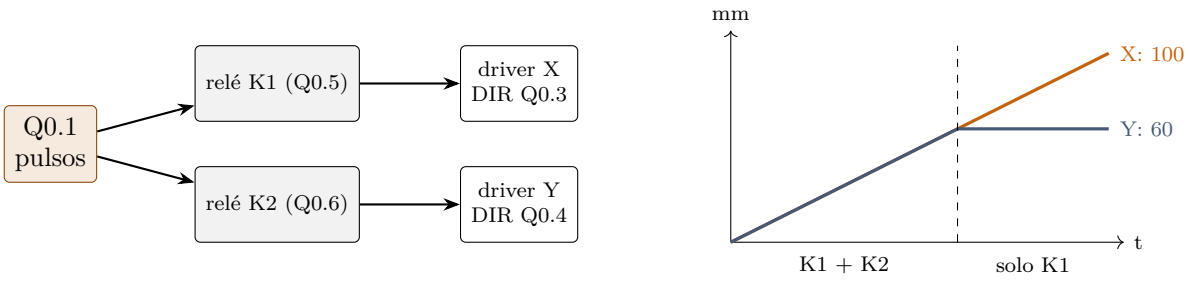


Figura 16: Q0.1 compartido: X e Y avanzan juntos a la misma velocidad durante el tramo común; luego se abre el relé del eje que llegó y el otro sigue solo.

El programa reparte cada movimiento en un **tramo común** (los dos relés cerrados) y un **resto** (solo el eje más largo). La base va por su propio tren y se mueve a la vez.

7. Los modos de mando

La página ofrece tres formas de mandar el brazo. Las tres terminan en lo mismo: un destino para los tres ejes (θ, carro X, carro Y) que se convierte en pulsos. Lo que cambia es *cómo se elige ese destino*.

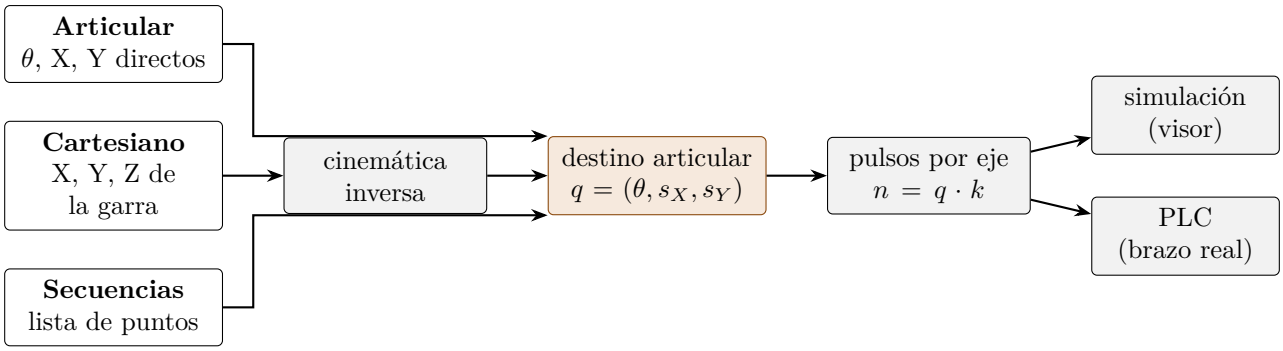


Figura 17: Los tres modos convergen en un destino articular.

La conversión a pulsos usa la transmisión: en X e Y, $k = 200$ pulsos/mm (1000 pulsos por vuelta / husillo de 5 mm); en la base, $k = 190{,}3$ pulsos/grado (valor calibrado en la planta).

7.1. Modo articular

Propósito. Mover cada eje por separado, como con los pulsadores del panel pero escribiendo el valor exacto. Es el modo más directo y el más seguro para probar la máquina: se sabe exactamente qué motor se mueve y cuánto.

Cómo funciona.

1. El usuario fija θ (grados), s_X y s_Y (mm) con los deslizadores o los números.

2. La página limita cada valor a su recorrido: $0 \le s_X, s_Y \le 170$ mm y $-175° \le \theta \le 175°$.
3. Con la **cinemática directa** calcula dónde quedará la garra (para mostrar X, Y, Z y dibujar el brazo).
4. *En simulación:* el visor interpola de la pose actual a la nueva con un perfil suave. Si está marcada la casilla del PTO compartido, imita el tramo común de X e Y.
5. *Con el brazo real:* la página envía `mover` con el destino; la app calcula los pulsos *relativos* a la posición contada por el PLC ($\Delta n = n_{destino} - n_{actual}$) y los escribe en el buzón. La página espera a que el PLC informe *reposo*.

A partir de qué se logró. De la relación lineal entre pulsos y desplazamiento de cada eje (paso del husillo, micropasos del driver, reducción de la base) y del programa nuevo del PLC, que cuenta los pulsos. La cinemática directa sale de la geometría del mecanismo:

$$r = u_A + d_g + L_9 \cos\varphi, \qquad z = h_A + L_9 \sin\varphi - h_g, \qquad X = r\cos\theta,\ Y = r\sin\theta,\ Z = z$$

donde u_A depende de s_X (el carro X desplaza todo el brazo) y el ángulo φ del brazo depende de s_Y a través del lazo de bielas A–B–C (se resuelve como la intersección de dos circunferencias).

Limitaciones.

- X e Y no pueden moverse a velocidades distintas: comparten Q0.1, así que en diagonal la garra no sigue una recta.
- No hay rampas de aceleración: por encima de 4–5 kHz los motores pueden perder pasos, y el PLC no lo detectaría (lazo abierto).
- El sentido de giro de la base es un SUPUESTO por confirmar.

7.2. Modo cartesiano

Propósito. Pedir *dónde* debe estar la garra (X, Y, Z en mm) sin pensar en los carros. Es lo que se necesita para tareas reales: tomar una pieza en un punto y dejarla en otro.

Cómo funciona: la cinemática inversa. Dado el punto (X, Y, Z), la página calcula los carros en cuatro pasos:

1. **Giro de la base**: $\theta = \text{atan2}(Y, X)$, y la distancia horizontal al eje $r = \sqrt{X^2 + Y^2}$.
2. **Ángulo del brazo** a partir de la altura: $\sin\varphi = \dfrac{Z + h_g - h_A}{L_9}$. Si el cociente pasa de 1, la altura es inalcanzable.
3. **Carro X** a partir del alcance: $s_X = r - d_g - L_9\cos\varphi - u_0$.
4. **Carro Y**: con $\psi = \varphi + \beta$ se conoce la posición del pasador B; el bloque del carro Y debe estar donde la biela de longitud L_{12} une B con C. Es la intersección de una circunferencia con una recta, y se elige la solución que corresponde al montaje real.

Luego se comprueba que los carros queden dentro de su recorrido y que la pinza no choque con la mesa.

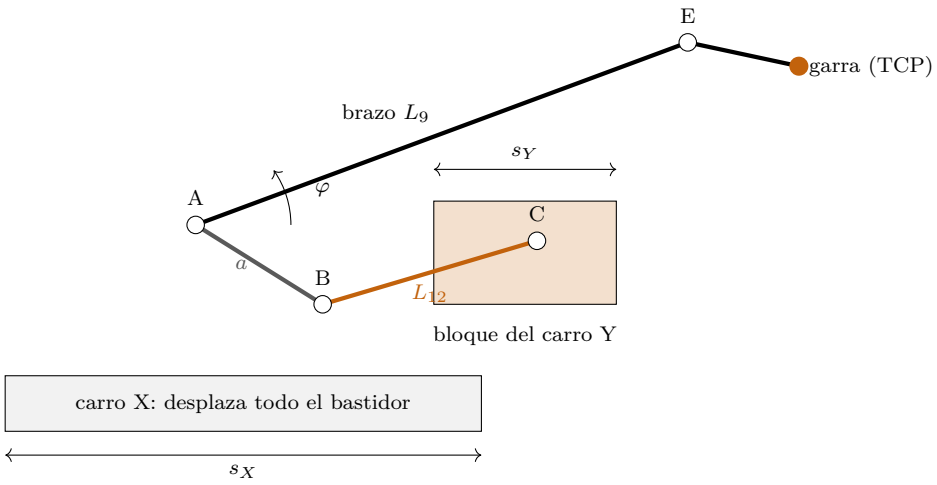


Figura 18: Esquema del mecanismo (no a escala). El carro Y empuja el bloque; la biela L_{12} hace girar el balancín A–B, que es parte del brazo; el ángulo φ del brazo fija la altura de la garra.

La trayectoria. En simulación, la página no salta al destino: divide la recta entre la garra y el objetivo en unos 20 o más puntos, con un perfil suave $w(t) = t^2(3 - 2t)$ (arranca y frena despacio), y calcula la inversa en cada uno. Si algún punto intermedio no es alcanzable, renuncia a la recta e interpola directamente los ejes. Con el brazo real se envía solo el destino final.

El espacio de trabajo. Como s_X y s_Y van de 0 a 170 mm, la garra solo puede estar dentro de la imagen de ese cuadrado, que en el plano vertical (r, z) es una **franja curva**:

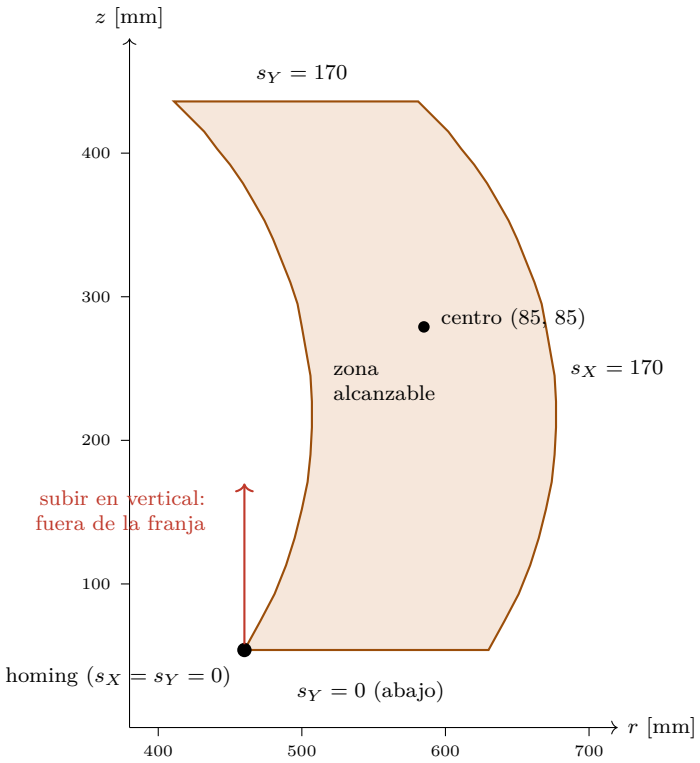


Figura 19: Corte vertical del espacio de trabajo (calculado con el modelo). Desde el homing, subir o bajar en vertical sale de la franja.

Por eso muchos puntos escritos a mano resultan «inalcanzables», sobre todo cerca del homing. La página ahora busca y propone el **punto alcanzable más cercano**: recorre los valores posibles de s_X y s_Y (con el θ del objetivo), calcula la directa de cada uno y se queda con el que tiene menor distancia al objetivo.

A partir de qué se logró. Del modelo geométrico del mecanismo (levantamiento de medidas y fotos), resolviendo las ecuaciones de lazo cerrado de las bielas. El **jacobiano** se calculó también para detectar posturas singulares, donde un movimiento pequeño de un carro casi no mueve la garra.

Limitaciones.

- Con el brazo real la trayectoria **no es recta**: el PLC recibe solo el destino y, por el tren de pulsos compartido, mueve X e Y a la misma velocidad.
- El espacio de trabajo es una franja estrecha: muchos puntos no son alcanzables y conviene moverse de a poco.
- La exactitud depende de las cotas: varias son estimadas o SUPUESTOS (por ejemplo, la posición de algunos pasadores). Un error de pocos milímetros en una biela desplaza la garra.
- El sistema es de lazo abierto: si un motor pierde pasos, el gemelo no lo sabe hasta el siguiente homing.

7.3. Modo secuencias

Propósito. Enseñar al brazo una tarea por puntos (por ejemplo: ir sobre la pieza, bajar, cerrar la garra, subir, llevarla, abrir) y repetirla. Es la versión moderna de las rutinas fijas del programa original, pero sin reprogramar el PLC.

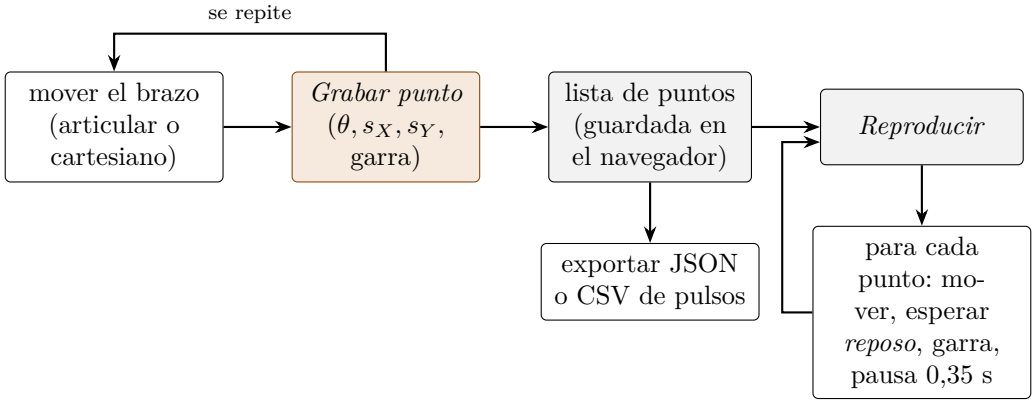


Figura 20: Grabar y reproducir una secuencia.

Cómo funciona.

- **Grabar** guarda la pose actual de los ejes y el estado de la garra. La lista se conserva en el almacenamiento del navegador.
- **Reproducir** recorre la lista: manda cada punto como un movimiento articular, *espera a que el PLC informe que terminó* (consultando el estado), acciona la garra si cambió y sigue con el siguiente. *Detener* corta la secuencia.
- **Exportar CSV de pulsos** genera una tabla con los pulsos absolutos y relativos de cada eje, las direcciones y el reparto del tren compartido (tramo común con K1 y K2, y tramo con un solo relé). Sirve para programar la secuencia directamente en el PLC, sin PC.

A partir de qué se logró. Del modo articular (cada punto es un destino articular), de la conversión a pulsos de la cinemática y del estado *reposo/moviendo* que informa el programa nuevo del PLC, que permite saber cuándo terminó cada punto.

Limitaciones.

- Los puntos se guardan en *ese* navegador: otro equipo no los ve, salvo que se exporten y se importen.
- Entre puntos el brazo se mueve en articular: no hay control de la trayectoria intermedia ni velocidad por punto.
- La secuencia la conduce la página: si se cierra la página o se pierde la red, la secuencia se detiene (el watchdog del PLC frena el movimiento en curso).
- No hay condiciones (por ejemplo, «esperar un sensor»): es una lista fija.

8. Seguridad por capas

Ninguna protección se apoya en una sola pieza. Si una capa falla, la siguiente sigue actuando.

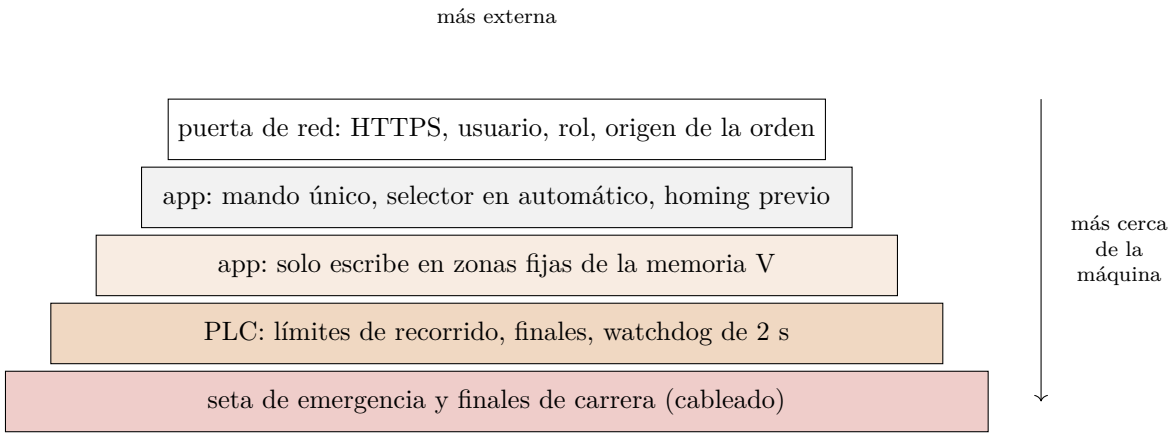


Figura 21: Capas de seguridad, de la más cercana al brazo a la más externa.

9. Limitaciones generales y siguientes pasos

Limitación	Cómo se podría resolver
Lazo abierto: los pasos perdidos no se detectan	Encoders en los ejes y comparación continua entre posición pedida y medida
Sin rampas de aceleración	Perfiles de velocidad en el PLC (PTO con varios segmentos)
X e Y comparten el tren de pulsos	Una salida de pulsos por eje (otro PLC o un módulo de posicionamiento)
Cotas estimadas o SUPUESTAS	Medirlas en la planta y ajustar `cinematica.js`
Acceso solo en la red local	VPN gratuita (por ejemplo Tailscale) o túnel
Comunicación a 9,6 kbit/s	Volver a 19,2 kbit/s en el bloque de sistema
Dependencia del cable USB-PPI	Probar un convertidor USB–RS485 genérico